N-gram language models

Class 3 of 24: Natural Language Processing

CSE 4811 / DS 4213 (Fall 2026)

Muhammad Anwarul Azim
Lecturer, Dept. of CSE
United International University

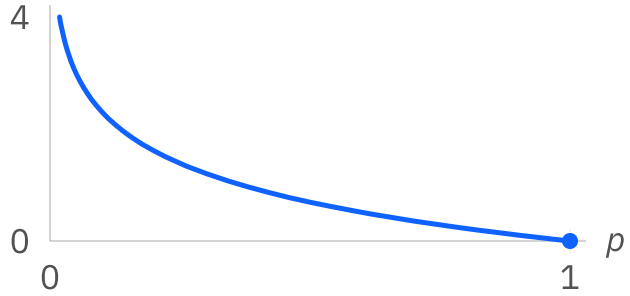
Recap

Naive Bayes, a model for classification

$$p(k \mid \text{text}) = \frac{p(\text{text} \mid k)\, p(k)}{\sum_j p(\text{text} \mid j)\, p(j)}$$

The loss

$$L(p) = -\log p$$



Naive Bayes can write

match	wins	star	match	wins	song	match	film	wins	match

Each row of its table can also write text, one word at a time.

Word probabilities

Probabilities	match	star	film	wins	song	Total
sports	0.40	0.20	0.05	0.30	0.05	**1**
entertainment	0.05	0.30	0.35	0.10	0.20	**1**

$$p(w \mid k) = \frac{\text{count}(w, k)}{\text{words in topic } k}$$

Dividing each count by its row total gives $p(w \mid k)$. Each row adds up to 1.

Classifying a headline

Test headline `Star film wins`

Probabilities	match	star	film	wins	song
sports	0.40	0.20	0.05	0.30	0.05
entertainment	0.05	0.30	0.35	0.10	0.20

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	0.20 × 0.05 × 0.30 = 0.003	0.6	0.0018	0.30
entertainment	0.30 × 0.35 × 0.10 = 0.0105	0.4	0.0042	**0.70**
Sum		**1**	**0.0060**	**1**

Posterior



Prediction: **entertainment**

The topic with the largest posterior is the prediction: entertainment, with 0.70.

How good was Naive Bayes?

Dev macro-F1, higher is better



Naive Bayes reaches 0.836. It classifies 666 articles wrongly, against 973.

What is missing?

1. Word order

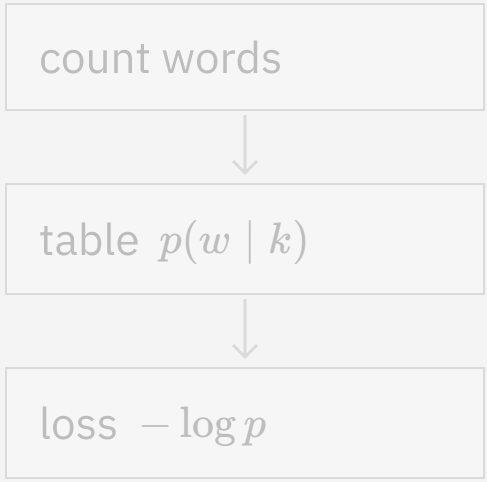
First

`Star film wins` ⟶ `film` `star` `wins`

`Wins film star` ⟶ `film` `star` `wins`

the same bag

2. Learning from the loss



no way back: the loss cannot change the counts

We first address word order.

Part 1: Word order

Each word, given the words before it.

How can we add order?

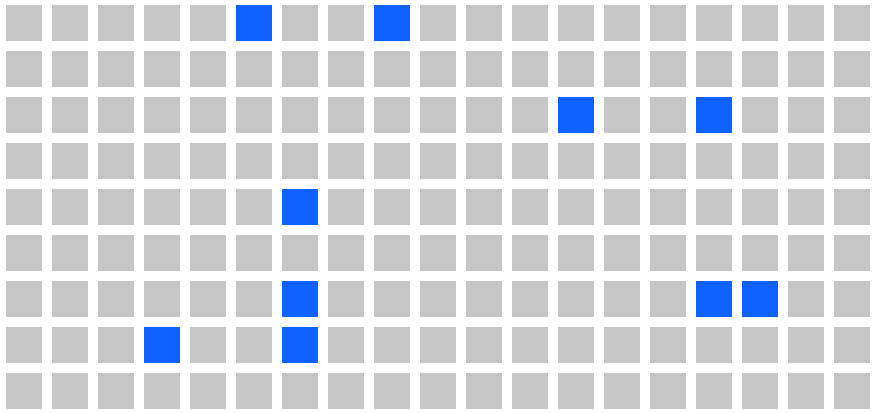
A dev headline `Soccer Star George Weah Wins Liberian Presidential Election`

soccer	$p(\text{soccer})$
soccer star	$p(\text{star} \mid \text{soccer})$
soccer star george	$p(\text{george} \mid \text{soccer star})$
soccer star george weah	$p(\text{weah} \mid \text{soccer star george})$
soccer star george weah wins	$p(\text{wins} \mid \text{soccer star george weah})$

The chain rule $p(w_1 \dots w_n) = \prod_{i=1}^{n} p(w_i \mid w_1 \dots w_{i-1})$

The chain rule of probability writes the probability of a text exactly as this product.

Conditional probability



`soccer`: 171 times

followed by `star`: 10 times

$$p(\text{star} \mid \text{soccer}) = \frac{\text{count}(\text{soccer star})}{\text{count}(\text{soccer})} = \frac{10}{171} = 0.058$$

$$p(\text{star}) = \frac{\text{count}(\text{star})}{\text{all words}} = \frac{489}{1{,}811{,}637} = 0.00027$$

Without the word before it, star is 489 of the 1,811,637 words: a probability of 0.00027.

Can we count every history?

History	Count in the training texts
`soccer`	171
`soccer star`	10
`soccer star george`	**0**
`soccer star george weah`	**0**
`soccer star george weah wins`	**0**

$$p(\text{george} \mid \text{soccer star}) = \frac{0}{10} = 0 \qquad p(\text{weah} \mid \text{soccer star george}) = \frac{0}{0}$$

Problem: most long histories occur rarely or never in the training texts, so their probabilities are 0 or 0/0.

The Markov assumption

soccer star george weah wins liberian **presidential** **election**

count 185

$$p(\text{election} \mid \text{soccer star george weah wins liberian presidential})$$

$$\approx p(\text{election} \mid \text{presidential})$$

$$p(w_i \mid w_1 \dots w_{i-1}) \approx p(w_i \mid w_{i-N+1} \dots w_{i-1})$$

$$p(\text{election} \mid \text{presidential}) = \frac{\text{count}(\text{presidential election})}{\text{count}(\text{presidential})} = \frac{31}{185} = 0.17$$

Solved: a short history occurs often enough in the training texts to be counted.

Bigrams, trigrams and so on

Unigrams 1 word	`soccer` `star` `george` `weah` `wins`
Bigrams 2 words	`soccer star` `star george` `george weah` `weah wins`
Trigrams 3 words	`soccer star george` `star george weah` `george weah wins`
4-grams 4 words	`soccer star george weah` `star george weah wins`
5-gram 5 words	`soccer star george weah wins`

A text of five words has five unigrams, four bigrams, three trigrams, two 4-grams and one 5-gram.

Part 2: The bigram model

One table per topic: each word, given the word before it.

From a row to a table

Word counts	match	star	film	wins	song	Total
sports	8	4	1	6	1	20
entertainment	1	6	7	2	4	20

sports

	match	star	film	wins	song	</s>	Total
<s>	3	1	0	1	1	0	**6**
match	0	2	0	1	0	5	**8**
star	0	0	1	3	0	0	**4**
film	0	0	0	1	0	0	**1**
wins	5	0	0	0	0	1	**6**
song	0	1	0	0	0	0	**1**

entertainment

	match	star	film	wins	song	</s>	Total
<s>	0	2	1	0	1	0	**4**
match	0	0	0	0	0	1	**1**
star	0	0	4	0	2	0	**6**
film	1	2	0	2	1	1	**7**
wins	0	0	0	0	0	2	**2**
song	0	2	2	0	0	0	**4**

Entertainment gets its own table. There, star is followed by film 4 times; in sports, once.

Counts to probabilities

- sports

	match	star	film	wins	song	</s>	**Total**
<s>	3/6	1/6	0	1/6	1/6	0	**1**
match	0	2/8	0	1/8	0	5/8	**1**
star	0	0	1/4	3/4	0	0	**1**
film	0	0	0	1/1	0	0	**1**
wins	5/6	0	0	0	0	1/6	**1**
song	0	1/1	0	0	0	0	**1**

- entertainment

	match	star	film	wins	song	</s>	**Total**
<s>	0	2/4	1/4	0	1/4	0	**1**
match	0	0	0	0	0	1/1	**1**
star	0	0	4/6	0	2/6	0	**1**
film	1/7	2/7	0	2/7	1/7	1/7	**1**
wins	0	0	0	0	0	2/2	**1**
song	0	2/4	2/4	0	0	0	**1**

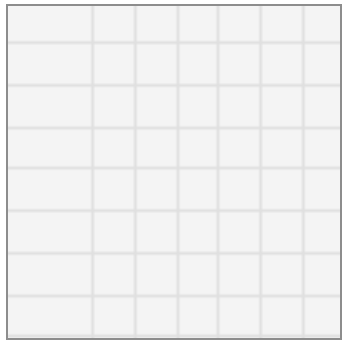
$$p(w \mid v, k) = \frac{\text{count}(v\, w, k)}{\text{count}(v, k)}$$

- $p(\text{film} \mid \text{star}) = 4/6$
- $p(\text{star} \mid \text{film}) = 2/7$

Order matters: in entertainment, p(film | star) = 4/6, but p(star | film) = 2/7.

Six topics, six tables

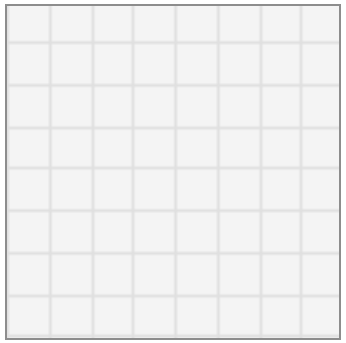
Each table: 110,609 rows × 110,609 columns ≈ 1.2×10^{10} cells



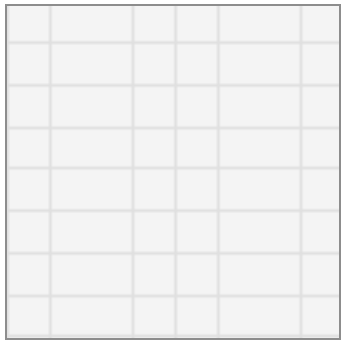
■ international



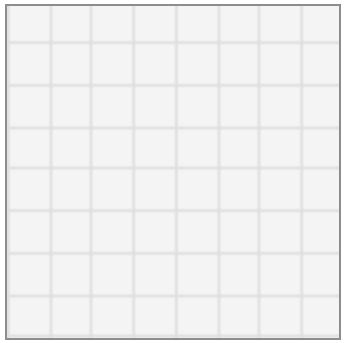
■ national politics



■ business and economy



■ entertainment



■ sports



■ science and technology

Different pairs in the training articles of each topic

205,294	186,068	202,177	202,436	198,696	189,218

The training articles of a topic contain about 200,000 different pairs. Only those cells are not 0.

The likelihood

Test headline `Star film wins`

$$p(\text{text} \mid k) = \prod_i p(w_i \mid w_{i-1}, k)$$

`<s> star` `star film` `film wins` `wins </s>`

■ sports

	match	star	film	wins	song	</s>
<s>	3/6	1/6	0	1/6	1/6	0
match	0	2/8	0	1/8	0	5/8
star	0	0	1/4	3/4	0	0
film	0	0	0	1/1	0	0
wins	5/6	0	0	0	0	1/6
song	0	1/1	0	0	0	0

■ entertainment

	match	star	film	wins	song	</s>
<s>	0	2/4	1/4	0	1/4	0
match	0	0	0	0	0	1/1
star	0	0	4/6	0	2/6	0
film	1/7	2/7	0	2/7	1/7	1/7
wins	0	0	0	0	0	2/2
song	0	2/4	2/4	0	0	0

- $p(\text{text} \mid \text{sports}) = 1/6 \times 1/4 \times 1/1 \times 1/6 = 1/144 \approx 0.0069$
- $p(\text{text} \mid \text{entertainment}) = 2/4 \times 4/6 \times 2/7 \times 2/2 = 2/21 \approx 0.095$

Entertainment is 13.7 times as likely as sports to write this headline. With single words, it was 3.5 times.

The posterior

Test headline `Star film wins`

Topic	Likelihood	Prior	Likelihood × prior	Posterior
sports	1/144	0.6	0.0042	0.0042 / 0.0423 = **0.10**
entertainment	2/21	0.4	0.0381	0.0381 / 0.0423 = **0.90**
Sum		**1**	**0.0423**	**1**



With pairs of words, the posterior of entertainment rises from 0.70 to 0.90.

A dev article

> A dev article, true topic: international
>
> `Soccer Star George Weah Wins Liberian Presidential Election His election marks the country's first democratic transition in over 70 years.`

Single words

- national politics **0.584** wrong

Pairs of words

- international **1.000** right

Word or pair	in international	in national politics
presidential	44	111
democratic	22	135
presidential election	20	6
democratic transition	0	0

As a pair, presidential election is more common in international, and democratic transition occurs in neither topic.

A misclassified article

> A dev article, true topic: business and economy
>
> `Markets Tumble Amid Brexit Chaos The vote to leave the EU has sparked a panic among investors around the world.`

Single words

- international **0.943** wrong

Pairs of words

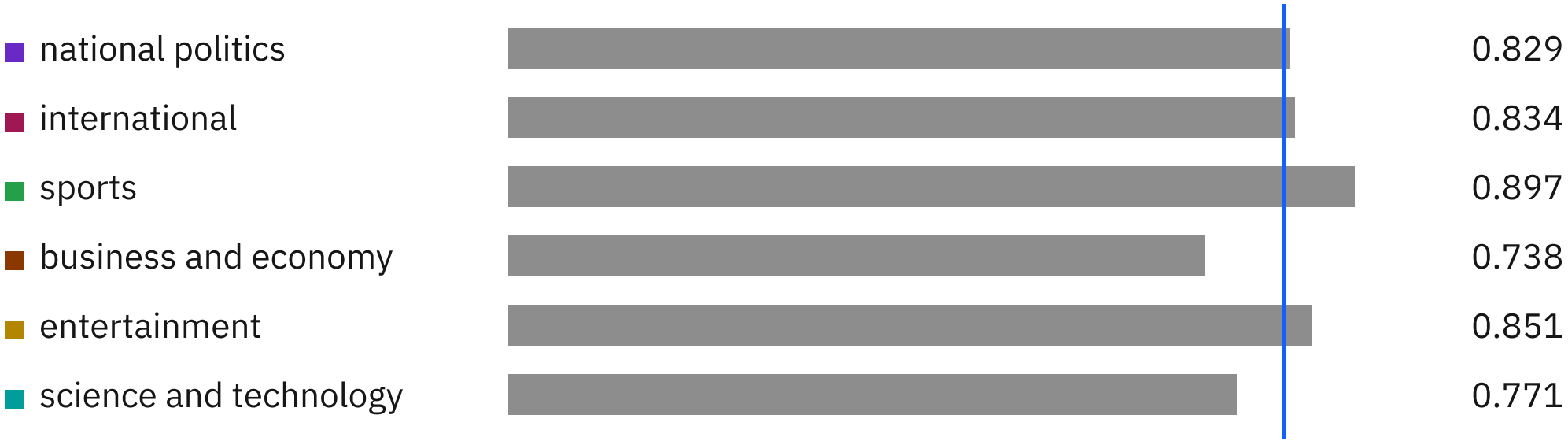
- international **0.999** wrong

Word or pair	in international	in business and economy
investors	1	53
markets	7	37
among investors	0	0
the eu	25	2
to leave	24	2
the vote	7	0

How good is the bigram model on the whole dev set?

Bigrams on the dev set

F1 for each topic



0.820 macro-F1, the mean of the six

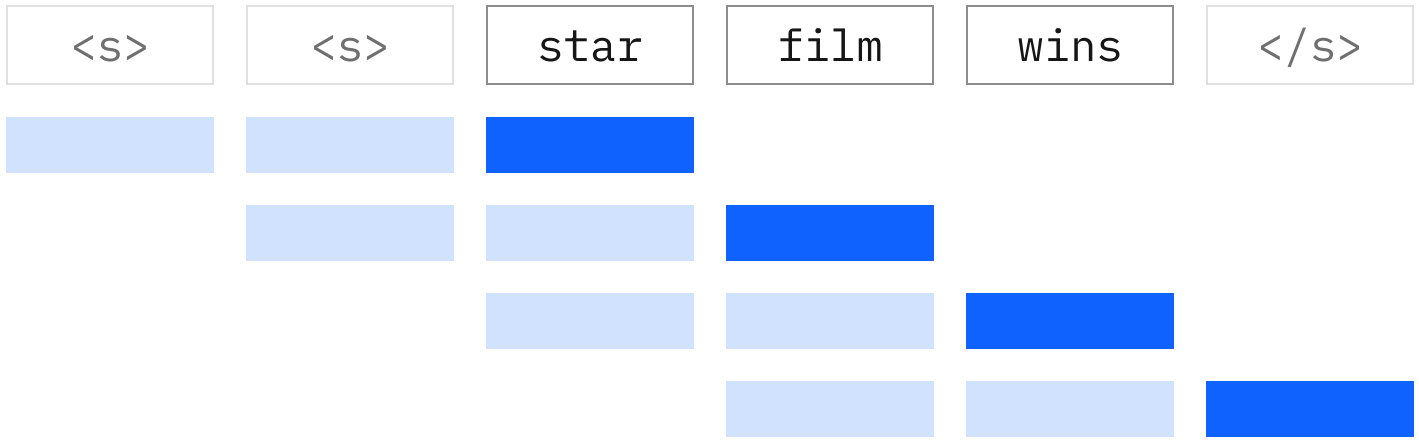
730 of 4,000 dev articles classified wrongly

Would a longer history help?

Part 3: More context

Two words of history for each word.

The trigram model



$$p(\text{star} \mid \text{<s> <s>})$$
$$p(\text{film} \mid \text{<s> star})$$
$$p(\text{wins} \mid \text{star film})$$
$$p(\text{</s>} \mid \text{film wins})$$

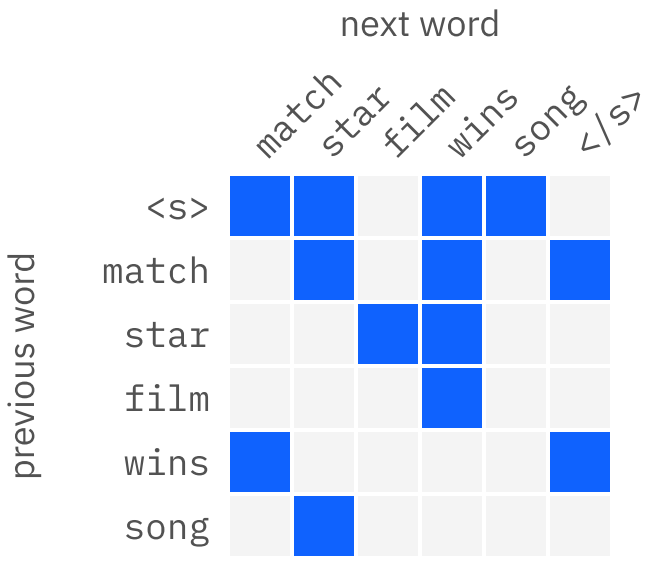
$$p(\text{text} \mid k) = \prod_i p(w_i \mid w_{i-2}\, w_{i-1}, k)$$

$$p(w \mid u\, v, k) = \frac{\text{count}(u\, v\, w, k)}{\text{count}(u\, v, k)}$$

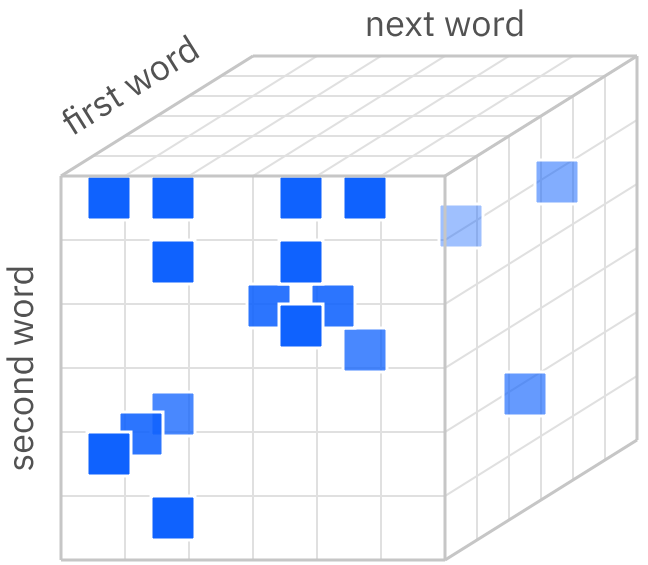
prior × likelihood → Bayes' formula → prediction

The prior, Bayes' formula and the prediction stay as before.

From a table to a cube



6 × 6 = 36 cells, 13 not 0



6 × 6 × 6 = 216 cells, 17 not 0

C1 data, each topic	Cells	Cells not 0	Cells not 0, about
Word list	110,608	33,313 to 37,309	1 in 3
Pair table	$110{,}609^2 = 1.2 \times 10^{10}$	186,068 to 205,294	1 in 60,000
Cube	$110{,}609^3 = 1.4 \times 10^{15}$	257,585 to 290,042	1 in 5 billion

Problem: almost every cell of the cube is 0. Of the dev trigrams, 84.4% never occur in the training texts of their topic.

Three models in code

Unigram

```
for words, k in zip(train_words, y):
    for w in words:
        count[k][w] += 1
        total[k] += 1
p = (count[k][w] + 1) / (total[k] + V)
```

Bigram

```
for words, k in zip(train_words, y):
    words = ["<s>"] + words + ["</s>"]
    for v, w in zip(words, words[1:]):
        count[k][v, w] += 1
        total[k][v] += 1
p = (count[k][v, w] + 1) / (total[k][v] + V)
```

Trigram

```
for words, k in zip(train_words, y):
    words = ["<s>", "<s>"] + words + ["</s>"]
    for u, v, w in zip(words, words[1:], words[2:]):
        count[k][u, v, w] += 1
        total[k][u, v] += 1
p = (count[k][u, v, w] + 1) / (total[k][u, v] + V)
```

All three add 1 to every count and V to every total, so an n-gram never seen still gets a small probability.

Results

Dev macro-F1, higher is better



Why does a longer history give a lower score?

How rare is a trigram?

> Markets Tumble Amid Brexit Chaos The vote to leave the EU has sparked a panic among investors around the world.

Count in the 40,000 training texts

Words	sparked 30	a 12,863	panic 18
Pairs	sparked a 5		
		a panic 1	
Trigram	sparked a panic 0		

The trigram itself never occurs.

The most common n-grams

Words		Pairs		Trigrams	
`the`	29,962	`of the`	2,735	`the u s`	358
`to`	15,877	`in the`	2,226	`one of the`	245
ও	12,976	করা হয়েছে	1,412	এ কথা বলেন	215
`of`	12,958	`it s`	1,213	বলে মন্তব্য করেছেন	199
`a`	12,863	`u s`	1,156	এসব কথা বলেন	186
Different words	**110,608**	Different pairs	**964,642**	Different trigrams	**1,496,818**
Seen only once	**45.4%**	Seen only once	**79.3%**	Seen only once	**92.9%**

Of all the different trigrams, 92.9% occur only once.

Unseen n-grams in an article

never occurs in the training texts

Words

markets | tumble | amid | brexit | chaos | the | vote | to | leave | the | eu | has | sparked | a | panic | among | investors | around | the | world

Pairs

markets tumble | tumble amid | amid brexit | brexit chaos | chaos the | the vote | vote to | to leave | leave the | the eu | eu has | has sparked | sparked a | a panic | panic among | among investors | investors around | around the | the world

Trigrams

markets tumble amid | tumble amid brexit | amid brexit chaos | brexit chaos the | chaos the vote | the vote to | vote to leave | to leave the | leave the eu | the eu has | eu has sparked | has sparked a | sparked a panic | a panic among | panic among investors | among investors around | investors around the | around the world

Share of all dev n-grams never seen in the training texts of their topic

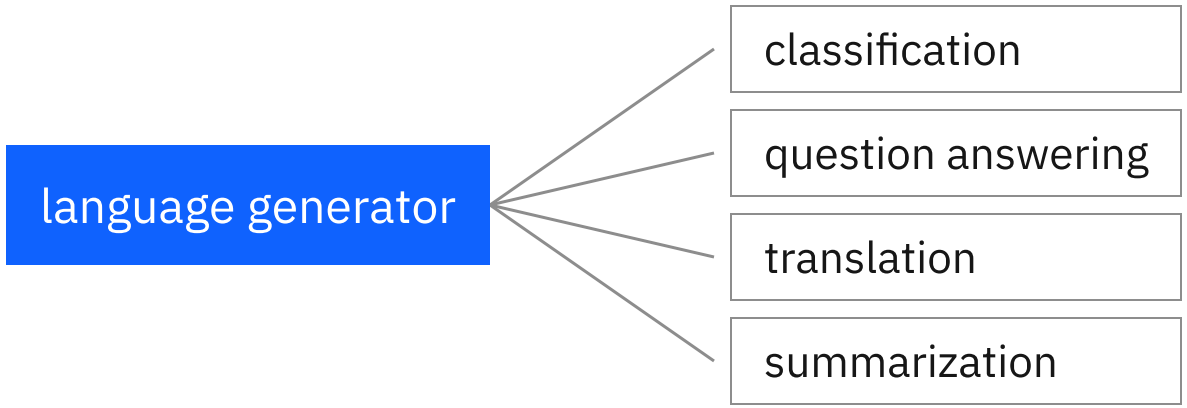
Words 3.5% Pairs 53.0% Trigrams 84.4%

Problem: an n-gram that never occurs gets almost the same probability in every topic, so it does not help to choose the topic.

Part 4: Generation

Writing text from the same counts.

Generation



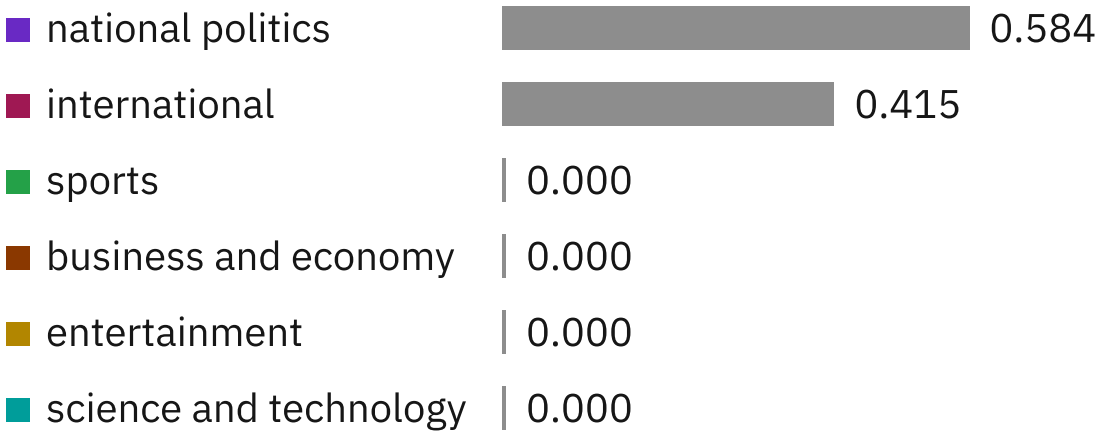
Unigram	$p(w \mid k)$
Bigram	$p(w \mid v, k)$
Trigram	$p(w \mid u\, v, k)$
A large language model	$p(\text{next token} \mid \text{all the words so far})$

Generation is the more important task, and generation needs word order.

Choosing the next word

Classification

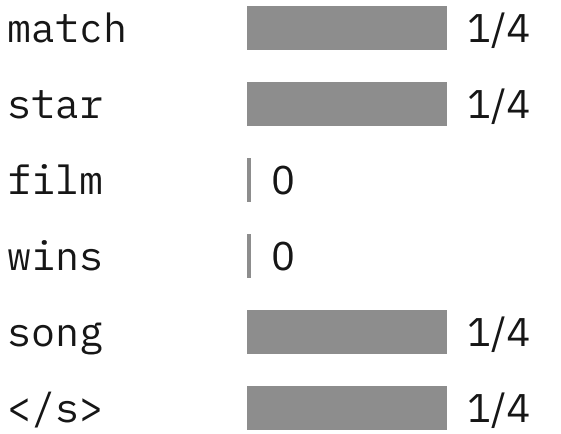
> Soccer Star George Weah Wins Liberian Presidential Election His election marks the country's first democratic transition in over 70 years.



Generation

entertainment

`star` `film` ?



110,609
classes for the next word in C1

Generation is classification of the next word, with one class for each word.

Three models write

Topic	Unigram	Bigram	Trigram
business and economy	colts সাংবাদিকদের দিলেন plays অবশেষে a s happiness wants ২২ babies owned কিংবদন্তী ইউরোপিয়ান স্থানীয় পারছে বিভাগীয় the can a	year s olympic gold medal win over here </s>	টস জিতে ব্যাটিং নেয়া বাংলাদেশ শুরুতেই বিপর্যয়ে পড়ে রানের খাতা খুলার আগেই ফিরে গেছেন অভিষিক্ত সাইফ হাসান এবার ইনজুরিতে পড়লেন
entertainment			
international			
national politics			
science and technology			
sports			

Each model writes from the counts of the chosen topic, without adding 1.

How good is a writer?

Mean loss per word

$$\text{loss} = -\frac{1}{n}\sum_{i=1}^{n} \log p(w_i \mid \text{history})$$

Perplexity

$$\text{perplexity} = e^{\,\text{loss}}$$

Toy example

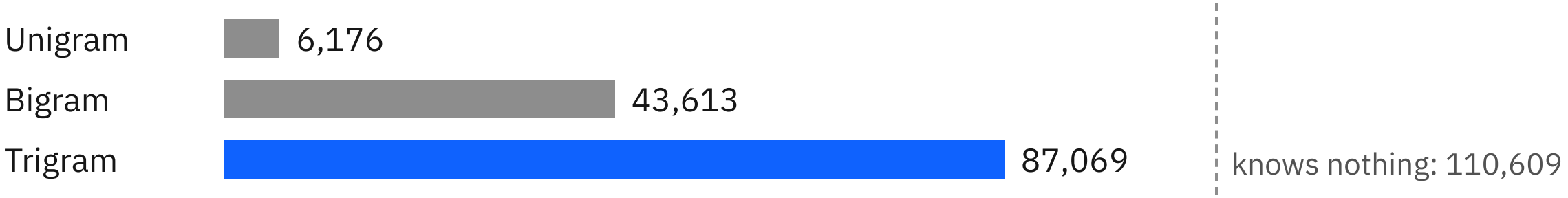
- `star` `film` → `song` $p = 1/4, \quad \text{loss} = \log 4 = 1.39, \quad \text{perplexity} = 4$

After star film, the model chooses among four words with probability 1/4 each, so its perplexity is 4.

What does perplexity mean?

Toy example	4 next words, 1/4 each	4
Knows nothing	110,609 next words, 1/110,609 each	110,609

Perplexity on the dev articles, C1 models with 1 added to every count



The trigram model writes the best text. Why does it give new text the highest perplexity?

Part 5: Unseen n-grams

Every n-gram needs a probability above 0.

A new headline

Test headline `Film star wins`

`<s> film` | `film star` | `star wins` | `wins </s>`

- sports

	match	star	film	wins	song	</s>
<s>	3/6	1/6	**0**	1/6	1/6	0
match	0	2/8	0	1/8	0	5/8
star	0	0	1/4	3/4	0	0
film	0	**0**	0	1/1	0	0
wins	5/6	0	0	0	0	1/6
song	0	1/1	0	0	0	0

- entertainment

	match	star	film	wins	song	</s>
<s>	0	2/4	1/4	0	1/4	0
match	0	0	0	0	0	1/1
star	0	0	4/6	**0**	2/6	0
film	1/7	2/7	0	2/7	1/7	1/7
wins	0	0	0	0	0	2/2
song	0	2/4	2/4	0	0	0

- $p(\text{text} \mid \text{entertainment}) = 1/4 \times 2/7 \times 0 \times 2/2 = 0$
- $p(\text{text} \mid \text{sports}) = 0 \times 0 \times 3/4 \times 1/6 = 0$

Loss $-\log 0 = \infty$

Problem: a single pair that never occurs in training makes the probability of the whole text 0.

Add-one smoothing

- entertainment

	match	star	film	wins	song	</s>	**Total**
	+1	**+1**	**+1**	**+1**	**+1**	**+1**	**+6**
star	1	1	5	1	3	1	**12**
$p(w \mid \text{star})$	1/12	1/12	5/12	1/12	3/12	1/12	**1**

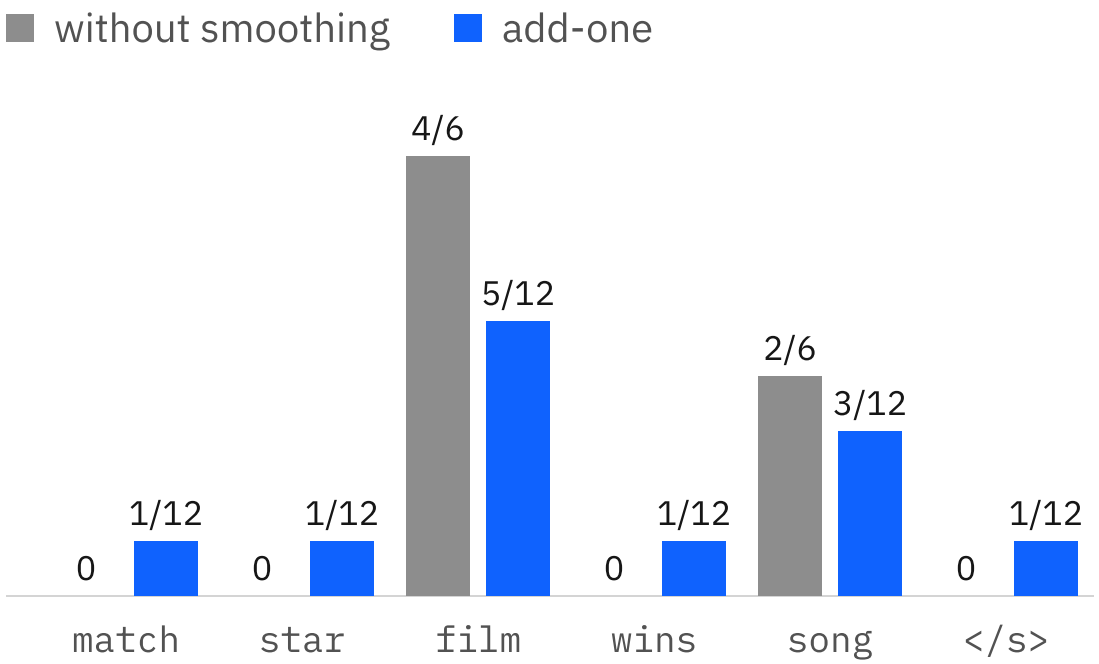
$$p(w \mid v) = \frac{\text{count}(v\,w) + 1}{\text{count}(v) + V}$$

$$p(\text{wins} \mid \text{star}) = \frac{0 + 1}{6 + 6} = \frac{1}{12}$$

Solved: $p(\text{wins} \mid \text{star}) = 1/12$, so the headline Film star wins gets a probability above 0.

The cost of add-one

$p(w \mid \text{star})$ in entertainment



Dev loss per word with add-one, lower is better



Problem: in C1, add-one adds $V = 110{,}609$ to every row total, so it gives too much probability to pairs that never occur.

What does adding 1 do?

The row of `presidential`, counted in all 40,000 training texts

185 real counts +1 for each of the 110,609 next words



110,794

Without smoothing $$p(\text{election} \mid \text{presidential}) = \frac{31}{185} = 0.17$$

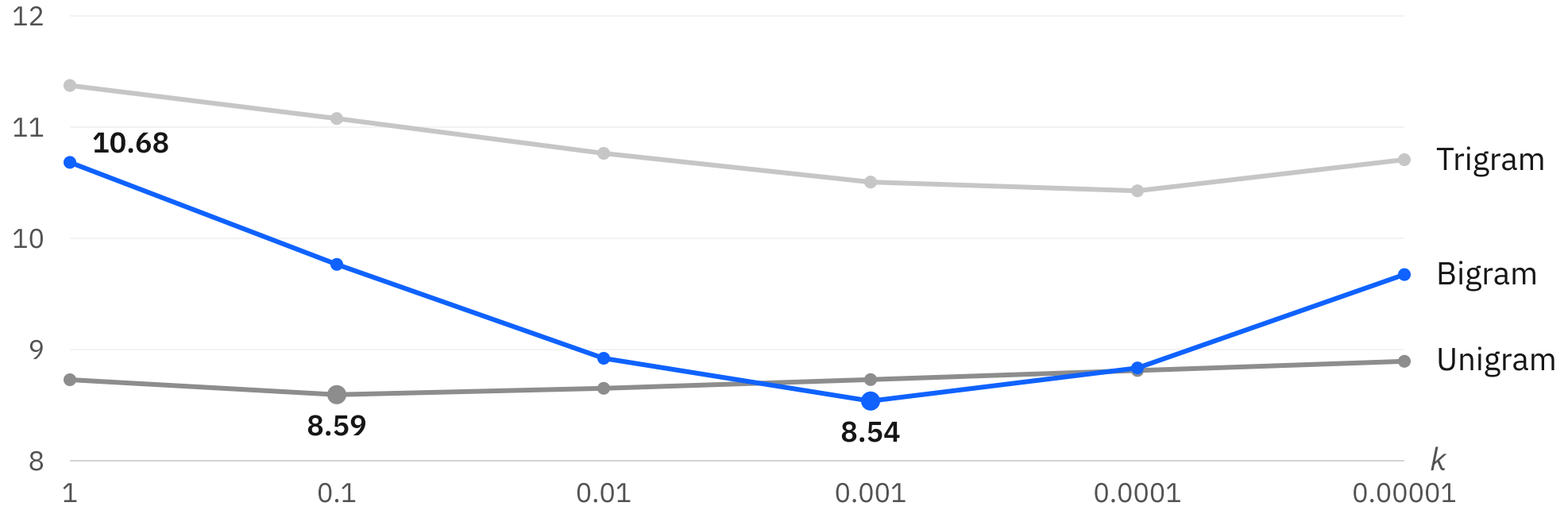
With add-one $$p(\text{election} \mid \text{presidential}) = \frac{31+1}{185+110{,}609} = 0.00029$$ **580 times smaller**

Problem: words never seen after presidential now get 99.8% of the probability.

Add-k smoothing

$$p(w \mid v) = \frac{\text{count}(v\,w) + k}{\text{count}(v) + kV}$$

Dev loss per word, lower is better



Solved: with $k = 0.001$, the bigram model predicts the dev text better than the unigram model.

Backoff

> The vote to leave the EU has sparked a panic among investors around the world.

		Count in the training texts
Trigram	panic among investors	0
Bigram	among investors	0
Unigram	**Add-one** 1/110,609 = 0.0000090 for every next word **Backoff** 64/1,811,637 = 0.000035 for investors	investors 64

Interpolation instead always mixes the trigram, bigram and unigram probabilities, with weights that add up to 1.

Handling unseen n-grams

Method	What it does	Its weakness
Add-one	Adds 1 to every count	Gives far too much to unseen n-grams: 99.8% of the row after presidential
Add-k	Adds a small k, chosen on the dev set	One k for every row. Bigram dev loss: 10.68 at k = 1, 8.54 at k = 0.001
Backoff	Uses the longest n-gram that was seen	Drops context. To add up to 1, the longer n-grams must give up some probability

All three only repair counts. None of them knows that two words are alike.

Part 6: Words as vectors

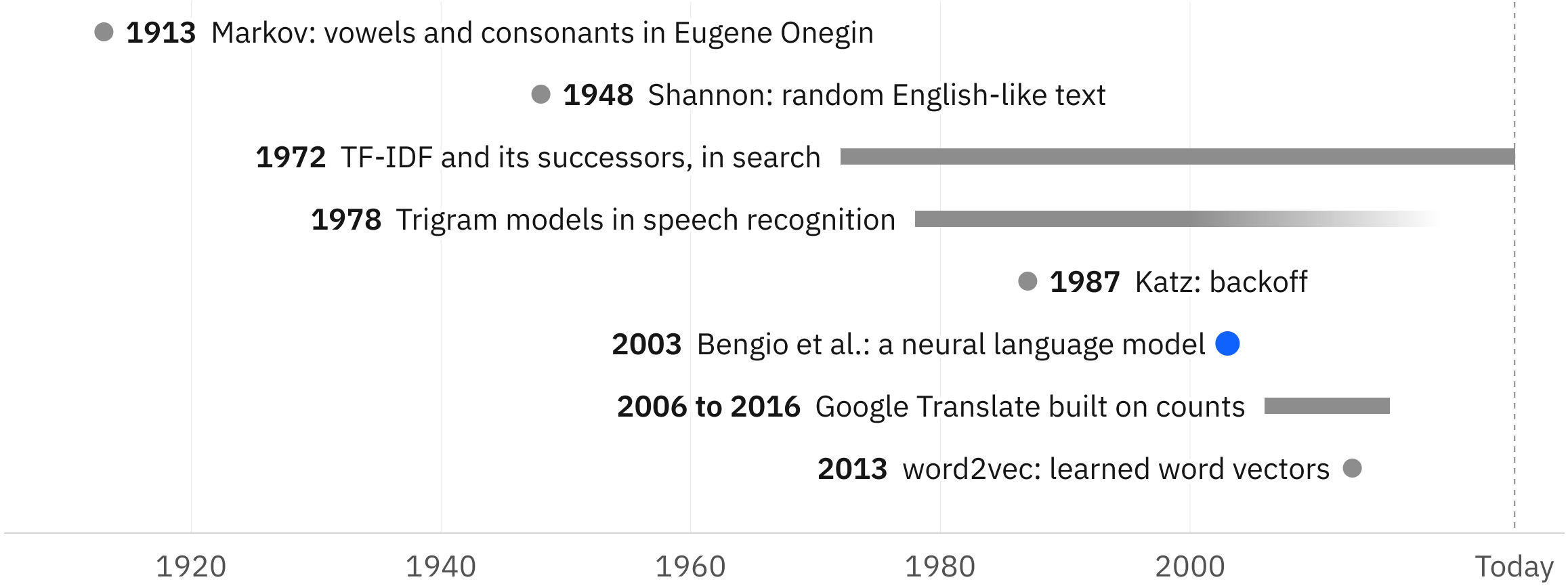
Each word becomes a vector, learned from the loss.

What can counting not do?

Cells in the table	$N = 1$	$N = 2$	$N = 3$
	$110{,}608$	$110{,}609^2 \approx 1.2 \times 10^{10}$	$110{,}609^3 \approx 1.4 \times 10^{15}$
Unseen n-grams	84.4% of the dev trigrams never occur in the training texts of their topic		
Backoff	`panic` `among` `investors`		

Problem: counting cannot use a long history, because most long n-grams never occur in the training texts.

How long did counting last?



So counting lasted about a hundred years, and search still uses it today.

Does the model know film and movie are alike?

Ten toy articles

```
match star film wins
star wins match
match star wins match
song star wins match
match wins match
```

```
wins match
star film song film wins
star film star song film wins
film star song star film
song star film match
```

Seven new articles

```
film star wins
star movie song movie wins
song star movie match
star movie song star movie
match star movie wins
song movie wins
star movie song film wins
```

	match	star	film	movie	wins	song	</s>	**Total**
film star	0	0	0	0	1	2	0	**3**
movie star	0	0	0	0	0	0	0	**0**

Test headline

Movie star wins

Which word comes after movie star?

The counts after movie star

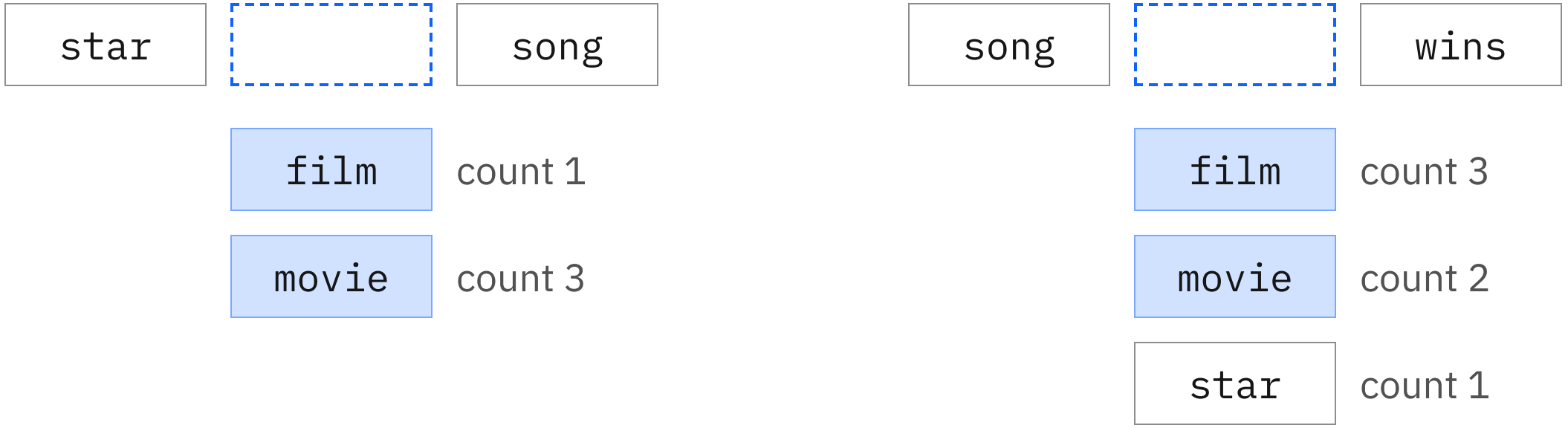
Test headline

`Movie star wins`

		match	star	film	movie	wins	song	</s>	**Total**	$p(\text{wins} \mid \text{history})$
Counts	film star	0	0	0	0	1	2	0	**3**	**1/3**
	movie star	0	0	0	0	0	0	0	**0**	**0/0**
Add-one	movie star	1	1	1	1	1	1	1	**7**	**1/7**
Backoff	movie star	0	0	5	6	4	2	0	**17**	**4/17**

Problem: what the counts learned about film star is never used for movie star, although film and movie are used alike.

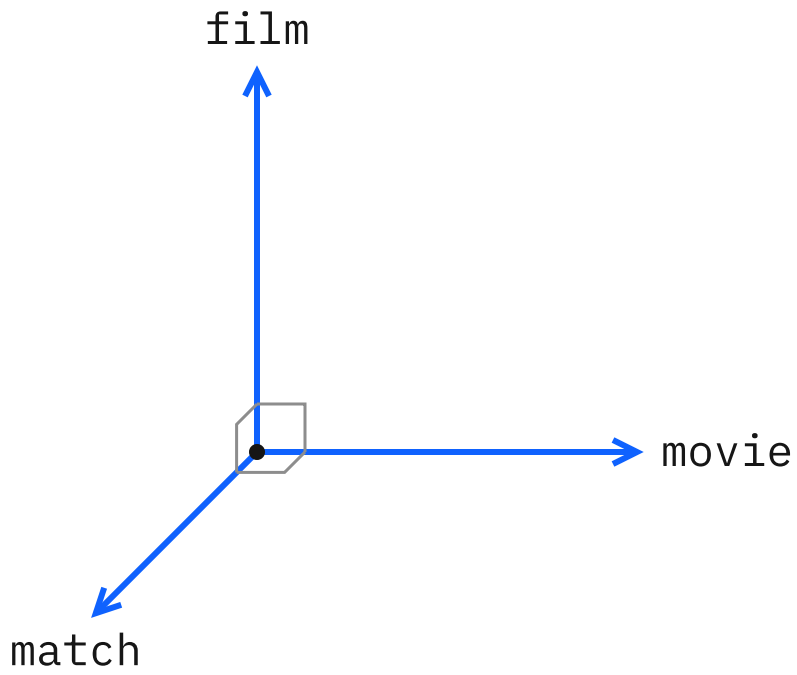
What makes two words alike?



Words that fit the same blanks are used alike, so we look at a word together with the words around it.

How is a word written now?

	match	star	film	movie	wins	song
film	0	0	1	0	0	0
movie	0	0	0	1	0	0
match	1	0	0	0	0	0

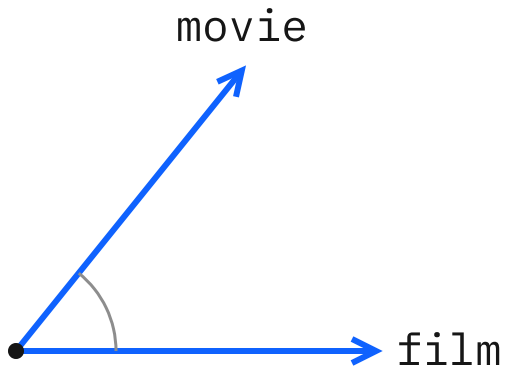


$$\cos(\text{film}, \text{movie}) = 0 \qquad \cos(\text{film}, \text{match}) = 0$$

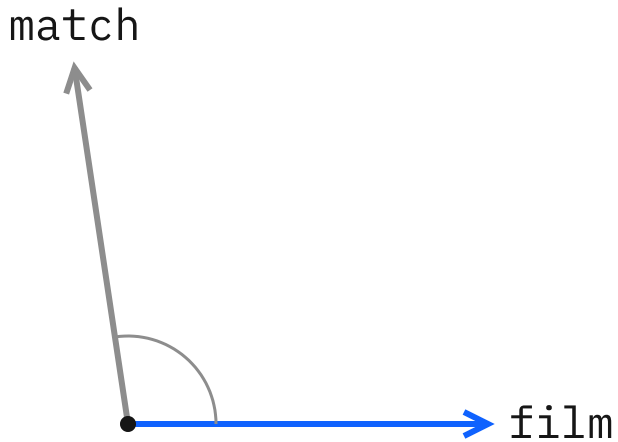
So film is as far from movie as it is from match.

A few numbers for each word

film	−1.21	−1.45	−2.53
movie	−2.20	0.62	−1.41
match	0.85	−1.69	0.96



$\cos(\text{film}, \text{movie}) = 0.63$



$\cos(\text{film}, \text{match}) = -0.15$

These numbers are not chosen by hand. They are learned from the 17 toy articles.

The paper

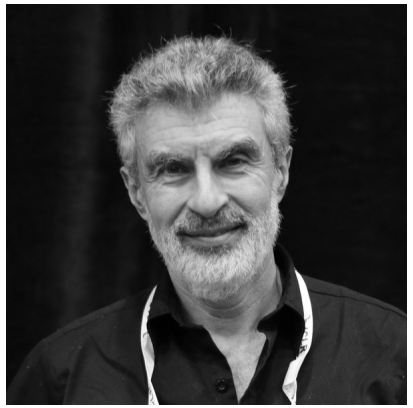
Journal of Machine Learning Research 3 (2003) 1137–1155 Submitted 4/02; Published 2/03

A Neural Probabilistic Language Model

Yoshua Bengio BENGIOY@IRO.UMONTREAL.CA
Réjean Ducharme DUCHARME@IRO.UMONTREAL.CA
Pascal Vincent VINCENTP@IRO.UMONTREAL.CA
Christian Jauvin JAUVINC@IRO.UMONTREAL.CA
Département d'Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal, Montréal, Québec, Canada

Three of the authors presented a shorter version in 2000 at NIPS, the conference now called NeurIPS.

Three researchers



Turing Award
2018

Yoshua Bengio

Université de Montréal and Mila

Neural language models and word vectors



Turing Award
2018

Nobel Prize
2024

Geoffrey Hinton

University of Toronto

Backpropagation, with Rumelhart and Williams, 1986



Turing Award
2018

Yann LeCun

New York University

Convolutional networks that read handwritten digits

In 2024 Hinton and John Hopfield shared the Nobel Prize in Physics, for discoveries that enable machine learning with neural networks.

Three ideas of the paper

A vector for each word `film` → −1.21 −1.45 −2.53

The next word from the vectors `movie` `star` → $p(\text{wins} \mid \text{movie star})$

Learned together Training text → Loss → Vectors and weights

Solved: words used alike can get nearby vectors, so what the model learns for film can also serve movie.

A vector for each word

E

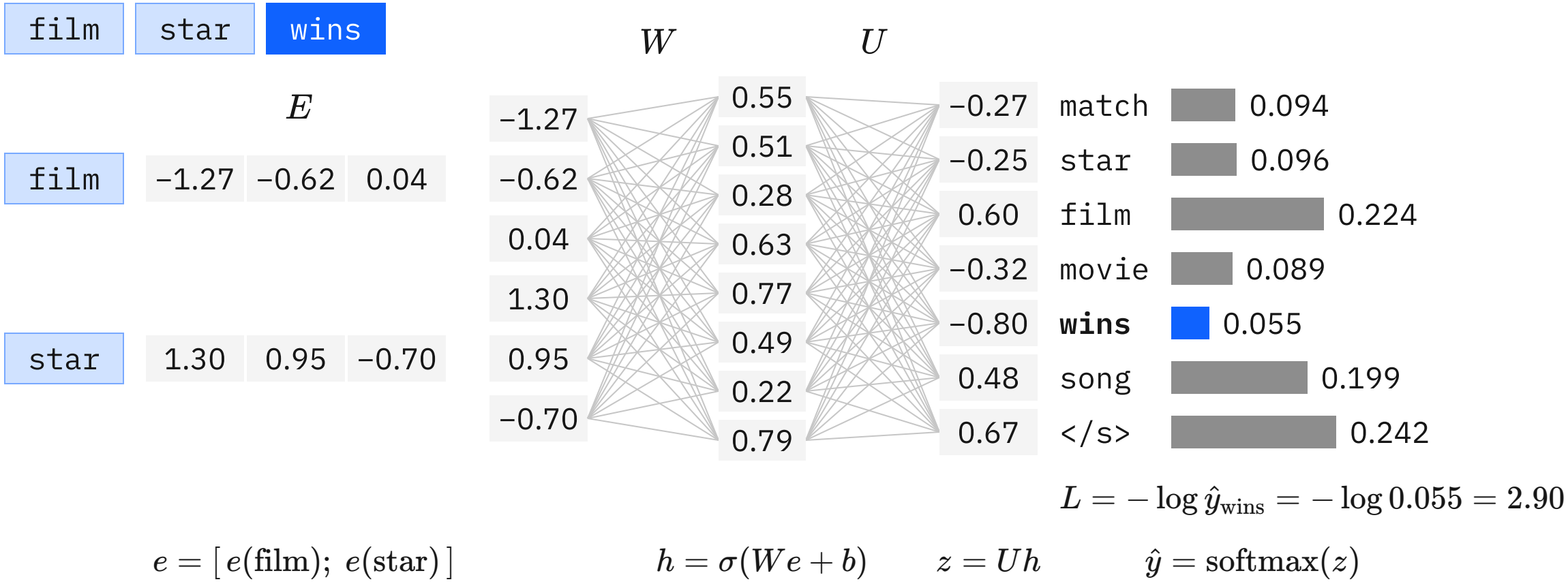
<s>	0.13	−0.13	0.64
match	0.10	−0.54	0.36
star	1.30	0.95	−0.70
film	−1.27	−0.62	0.04
movie	−2.33	−0.22	−1.25
wins	−0.73	−0.54	−0.32
song	0.41	1.04	−0.13

$$e(\text{film}) = [-1.27,\ -0.62,\ 0.04]$$



Training changes the numbers of E, so the points move.

The neural language model



The loss is $-\log \hat{y}$ for the true next word, wins: here 2.90.

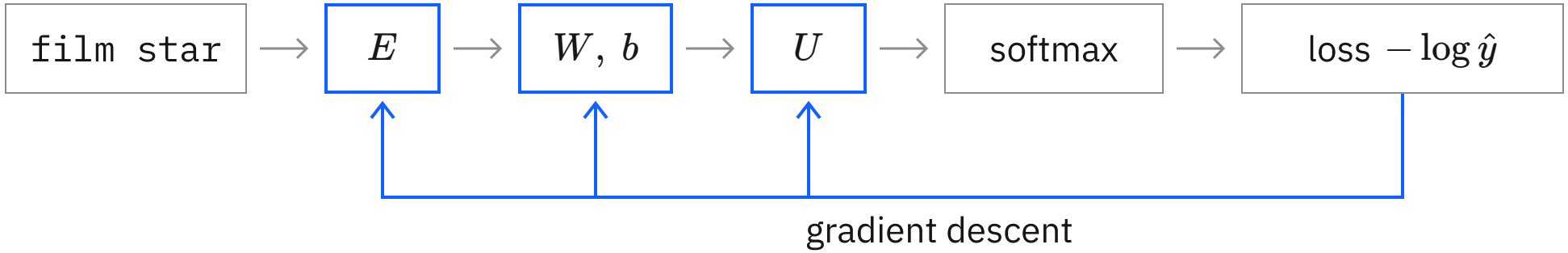
Training

Naive Bayes

count words → table $p(w \mid k)$ → loss $-\log p$

no way back: the loss cannot change the counts

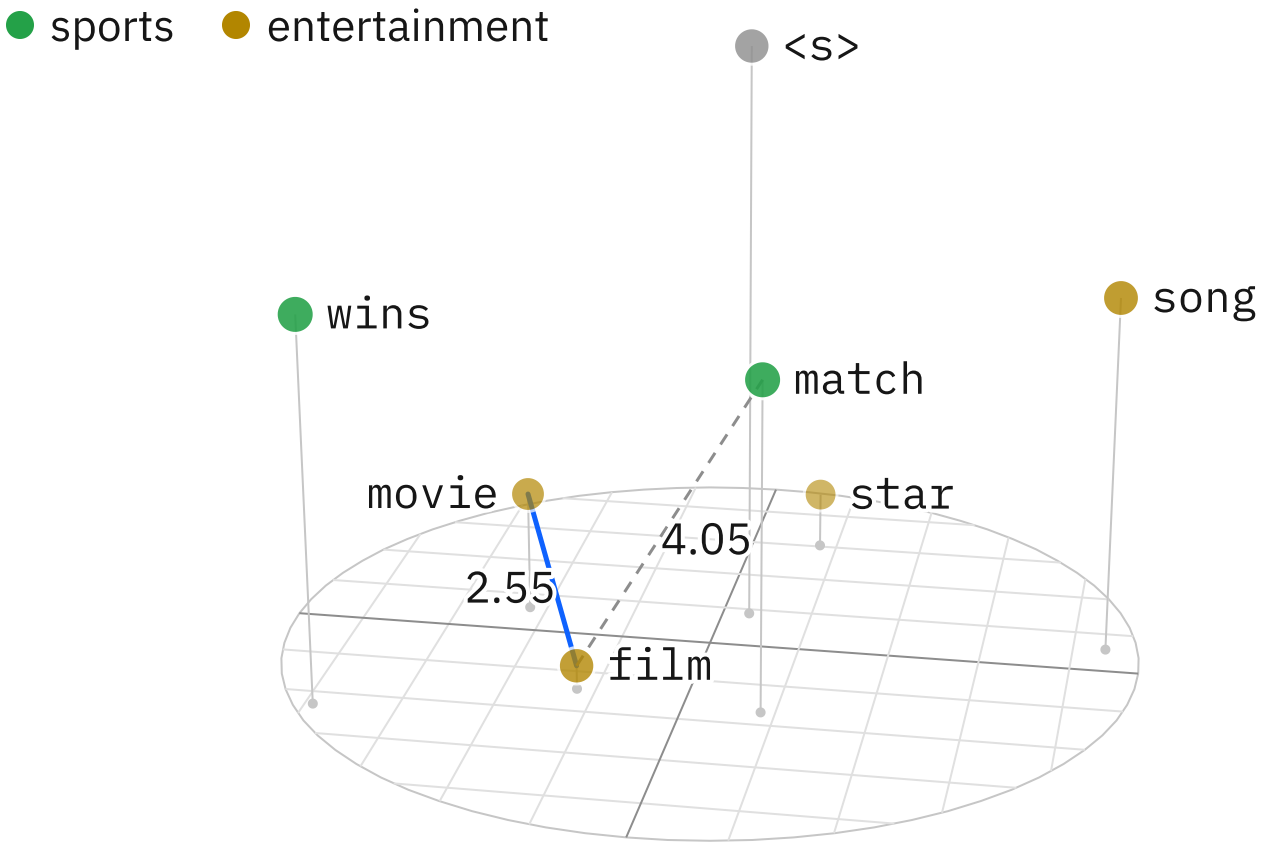
Neural language model

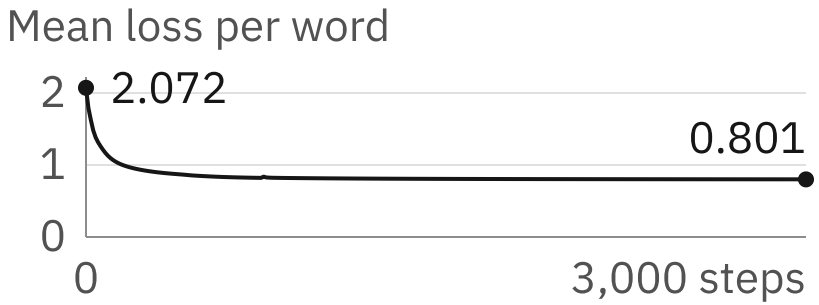


$\theta_{t+1} = \theta_t - \eta \nabla L$ for each of E, W, b and U

Solved: this model learns from its loss, which Naive Bayes could not do.

What is learned?





$p(\text{next word} \mid \text{history})$

	movie star	film star
match		
star	add-one: 1/7	
film		
movie		
wins	0.31	0.33
song	0.67	0.66
</s>		

Seen from every side, film and movie stay close, so what the model learns about film also applies to movie.

Counting and learning

	N-gram counts	Neural language model
Numbers to store	1.4×10^{15} cells for trigrams	$V \times m$ in E, plus W, b and U
Unseen history	add-one: 1/7 for each next word	close to its prediction after film star
Similar words	each word counted on its own	can give them nearby vectors
How it is made	counted from the training texts	learned by gradient descent
Window	fixed: the last $N - 1$ words	fixed: the last $N - 1$ words

Both models see only a fixed number of previous words. Class 4 removes this limit.

What we learned

Word order	The chain rule conditions each word on every word before it. The Markov assumption keeps only the last N – 1.	$p(w_i \mid w_1 \dots w_{i-1}) \approx p(w_i \mid w_{i-N+1} \dots w_{i-1})$
The bigram model	Each topic has a table of pairs, counted from its training texts.	$p(w \mid v, k) = \dfrac{\text{count}(v\,w, k)}{\text{count}(v, k)}$
More context	Longer histories are rarer, so the trigram model classifies worst.	Dev macro-F1: unigram 0.836, bigram 0.820, trigram 0.733
Generation	Generation chooses the next word. The counts write well, because they follow only n-grams that were seen.	$p(\text{next word} \mid \text{history})$
Unseen n-grams	Smoothing gives every n-gram a probability above 0. Backoff uses shorter histories.	$p(w \mid v) = \dfrac{\text{count}(v\,w) + k}{\text{count}(v) + kV}$
Words as vectors	A neural language model learns a vector for each word from the loss. Words used alike can get nearby vectors.	$h = \sigma(We + b), \quad \hat{y} = \text{softmax}(Uh)$

References

Textbook

Jurafsky, D. and Martin, J. H. (2026). *Speech and Language Processing*, 3rd ed. draft, August 2026. Chapter 3, sections 3.1–3.7, pp. 68–88 (n-gram language models: the chain rule, eq. 3.4; the Markov assumption, eqs. 3.7 and 3.8; probabilities from counts, eq. 3.11; Laplace smoothing, eq. 3.26; add-k smoothing, eq. 3.28; interpolation and backoff, sections 3.6.3 and 3.6.4). Chapter 6, section 6.5, pp. 163–168 (embeddings as input; the feedforward neural language model, pp. 165–168, eq. 6.24, fig. 6.14).

Data and code

The script c03_build.py (the class 1 tokenizer; the unigram, bigram and trigram models; add-k smoothing; the neural language model on the toy example), on the C1 training set (40,000 texts) and dev set (4,000 texts).

Paper

Bengio, Y., Ducharme, R., Vincent, P. and Jauvin, C. (2003). *A neural probabilistic language model.* Journal of Machine Learning Research 3, 1137–1155.

History

Markov (1913), on vowels and consonants in *Eugene Onegin*. English translation: Science in Context 19(4), 591–600, 2006. Shannon (1948). Bell System Technical Journal 27(3), 379–423. Spärck Jones (1972). Journal of Documentation 28(1), 11–21. Bahl et al. (1978). ICASSP 1978, 422–424. Katz (1987). IEEE Transactions on Acoustics, Speech, and Signal Processing 35(3), 400–401. Rumelhart, Hinton and Williams (1986). Nature 323, 533–536. Robertson et al. (1994), BM25. Okapi at TREC-3. Bengio, Ducharme and Vincent (2000). NIPS 13, 932–938. Mikolov et al. (2013). arXiv:1301.3781. Google blogs of 28 April 2006, 27 September 2016 and 25 April 2017. ACM Turing Award 2018; Nobel Prize in Physics 2024.

Photos

Yoshua Bengio: Xuthoria, 2025. CC BY-SA 4.0, via Wikimedia Commons, File:ICLR 2025 - Yoshua Bengio 03.jpg; cropped and made grayscale.

Geoffrey Hinton: Arthur Petron, 2024. CC BY-SA 4.0, via Wikimedia Commons, File:Geoffrey E. Hinton, 2024 Nobel Prize Laureate in Physics.jpg; cropped and made grayscale.

Yann LeCun: Jérémy Barande / Ecole polytechnique Université Paris-Saclay, 2018. CC BY-SA 2.0, via Wikimedia Commons, File:Yann LeCun (41208595340).jpg; cropped and made grayscale.

Thank you.